

A More on Reasoning with Soft Rules

Let σ be a signature as in first-order logic. An LP^{MLN} program Π is a finite set of weighted rules of the form:

$$w : A \leftarrow B \quad (1)$$

where A is a disjunction of atoms of σ , B is a conjunction of literals (atoms and negated atoms) of σ , and w is a real number or the symbol α .

When A is $\perp$ (the empty disjunction), the rule asserts that B should be false in the stable model. An LP^{MLN} rule (1) is called *soft* if w is a real number or *hard* if w is α . An LP^{MLN} program is *ground* if its rules contain no variables. An LP^{MLN} program Π that contains variables is identified with a ground LP^{MLN} program $gr_\sigma[\Pi]$, which is obtained from Π by replacing every variable with every ground term of σ . The weight of a ground rule in $gr_\sigma[\Pi]$ is the same as the weight of the corresponding rule in Π . By $\bar{\Pi}$ we denote the unweighted logic program obtained from Π , i.e., $\bar{\Pi} = \{R \mid w : R \in \Pi\}$.

For a ground LP^{MLN} program Π , Π_I denotes the set of rules $w : R$ in Π such that I satisfies R (denoted $I \models R$) and $\text{SM}[\Pi]$ denotes the set $\{I \mid I \text{ is a (deterministic) stable model of } \bar{\Pi}_I\}$. The (unnormalized) weight of I under Π is defined as follows:

$$W_\Pi(I) = \begin{cases} \exp(\sum_{w:R \in \Pi_I} w) & \text{if } I \in \text{SM}[\Pi]; \\ 0 & \text{otherwise.} \end{cases}$$

The probability of I under Π is the normalized weight defined as follows:

$$P_\Pi(I) = \lim_{\alpha \rightarrow \infty} \frac{W_\Pi(I)}{\sum_{J \in \text{SM}[\Pi]} W_\Pi(J)}.$$

In Answer Set programming (ASP), search problems are reduced to computing *stable models* (a.k.a. answer sets), a set of beliefs described by the program. In the case of a Horn program, the stable models coincide with the minimal models. LP^{MLN} programs are transformed to meet the needs of an ASP solver (Gebser et al., 2014; Lee et al., 2017).

B Rule Support

We designed an experiment to show the impact of increasing the number of overlapping rules on the same target predicate. The goal is to measure how often multiple rules are triggered for the same target triple.

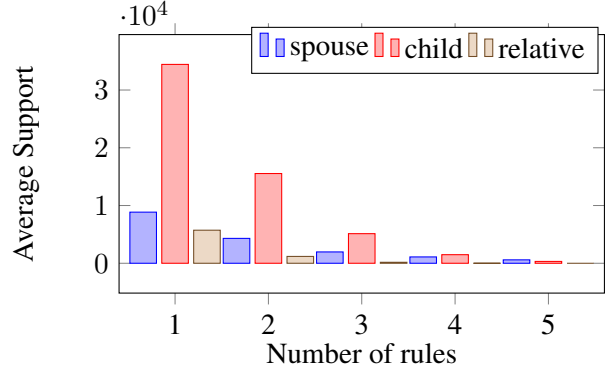


Figure 4: Support of the overlapping rules.

We measure this with the support of a rule, i.e., the number of triples in the knowledge base that satisfy all the atoms in the rule.

To compute the support for more than one rule, we combine the premises of the rules. In this experiment, we picked three predicates (*spouse*, *child* and *relative*), and for each one we selected ten rules randomly. Next, we used DBpedia online endpoint⁴ to compute the support for each combination of n ($n=1,2,\dots,5$) rules for each predicate. The results in Figure 4 show that by increasing the number of rules, the support decreases for all predicates. For combinations with more than three rules, the support is very small.

C More Experimental Details

For fine-tuning our models, we use Google Colaboratory (Bisong, 2019), which assigns random GPU clusters of various types. The number of parameters of our models is about 355M. We select the values of our hyper-parameters (shown in Table 8) on the development sets, by maximizing accuracy.

The execution times vary largely depending on the GPU at hand and on the scenario, with fine-tuning on a Tesla V100 taking from one hour for a single rule to a few hours for all the chaining experiments. The training/validation/testing splits are shown in Table 1. Table 9 shows the sizes of the used test datasets.

D Ablation

D.1 Impact of the Data Size

Setting. We report the impact of the size of the fine-tuning data on the model performance. As shown in Table 2, the accuracy of the fine-tuned model is higher for rules with higher confidence.

⁴<http://dbpedia.org/sparql>

Hyper-Parameter	Value
Learning Rate	1e-6
Weight Decay	0.1
Number of Epochs	3
Batch Size	16
Learning Rate Decay	Linear
Warmup Ratio	0.06

Table 8: Hyper-parameters for fine-tuning our model.

Dataset	Size
Mod0 Test(own)	2,667
Mod1 Test(own)	4,000
Mod2 Test(own)	5,334
Mod3 Test(own)	6,667
Mod4 Test(own)	8,000
Mod5 Test(own)	9,334
Test(D≤5)	9,334
Depth=0	16,057
Depth=1	6,608
Depth=2	5,389
Depth=3	3,993
Depth=4	2,619
Depth=5	1,336

Table 9: Number of examples in each of the test datasets for the chaining experiment.

We therefore divide the rules in three categories: *High* contains rules with confidence greater than 0.8, *Medium* has rules with confidence between 0.4 and 0.8, and *Low* is for the rest. There are six rules in the *Medium* category and the other two categories have five rules each. For each rule, we fine-tune seven models with 1k, 2k, 5k, 10k, 15k, 20k, and 30k examples.

Results. Figure 5 shows that having more training data improves the accuracy in all scenarios. For all categories, there is a sizable increase going from 10k to 15k examples; the impact is smaller for higher values. The highest increase is for rules with high confidence, and rules with medium confidence demonstrate larger increase than low confidence.

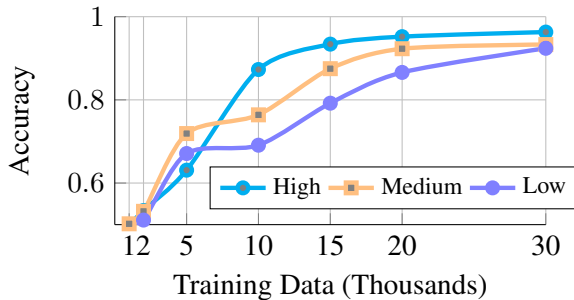


Figure 5: Impact of the training data size.

D.2 Role of the Example Format

Setting. When we teach rules to PLMs, we rely on examples with real names from a fixed pool. However, our goal is to teach PLMs the semantics of the soft rule, not the facts in our examples. Thus, we further design an experiment to assess the impact of the format used in the example facts on the behavior of the model. We distinguish two formats for the generated facts: (i) real names such as *Alice* and *IBM*, and (ii) letters such as *A* and *B*. We first use each format in fine-tuning and we then test both formats. We end up with two test/train scenarios: one with the same format and one with different formats. For this study, we use just one rule: $child(a,c) \wedge parent(c,b) \rightarrow spouse(a,b)$, with 30K examples for fine-tuning, and 2k for testing.

	Train Letter	Train Name
Test Letter	.981	.932
Test Name	.977	.985

Table 10: Impact of the example format on accuracy.

Results. The results in Table 10 show that the model performance does not depend heavily on using the same fact format for training and testing. With examples using letters in training, the results are slightly better in the case with two formats. We ultimately use names for testing and training in our default configuration as it yields better results.

E Impact of the Random Seed

Pre-trained transformers often suffer from instability of the results across multiple reruns with different random seeds. This usually happens with small training datasets (Dodge et al., 2020; Mosbach et al., 2021). In such cases, typically multiple reruns are performed, and the average value over these reruns is reported.

However, the numbers for the main experiments we report in this paper are not averaged over multiple reruns as our datasets are considerably large and the models did not suffer from instability due to random seeds. For example, when we reran RULE-BERT on a single-rule experiment three times, we obtained accuracy of 0.98959, 0.99551, 0.99636 with a standard deviation of only 0.003.

Yet, for the small dataset *bAbI*, we observed a much higher standard deviation of 0.17. Thus, in this case we report results that are averaged over ten reruns.

F Data Generation Example

We show an example of data generation for Algorithm 1. For simplicity, here we show an example of a hard rule, i.e., one whose confidence is implicitly set to one.⁵ We begin by setting the values of the input parameters:

Algorithm 1 Input:

- $r = \text{child}(A, C) \wedge \text{parent}(C, B) \rightarrow \text{spouse}(A, B)$
- $n = 8$
- $m = 5$
- $\text{pools} = \{\text{Alice}, \text{Bob}, \text{Carl}, \text{David}, \text{Eve}\}$

We set $n = 8$ to generate all the eight hypotheses. We start by generating a set of facts F (line 3), having predicates from the body of the rule with random polarity. We ensure that there are facts that trigger the rule. Their number should not exceed m . Here is an example of generated facts F :

Generated Facts F :

- f_1 : $\text{negparent}(\text{Eve}, \text{Carl})$
- f_2 : $\text{child}(\text{Eve}, \text{David})$
- f_3 : $\text{parent}(\text{Carl}, \text{Bob})$
- f_4 : $\text{child}(\text{Alice}, \text{Carl})$

Four facts are generated in total. Facts f_3 and f_4 trigger rule r . We then feed the rule r and facts F into the LP^{MLN} reasoner (line 4). The output O is then:

LP^{MLN} Reasoner Output O :

- o_1 : $\text{child}(\text{Eve}, \text{David})$
- o_2 : $\text{child}(\text{Alice}, \text{Carl})$
- o_3 : $\text{parent}(\text{Carl}, \text{Bob})$
- o_4 : $\text{spouse}(\text{Alice}, \text{Bob})$
- o_5 : $\text{negchild}(\text{Eve}, \text{Carl})$

We start generating the hypotheses:

Generated Hypotheses H :

- h_1 : $\text{child}(\text{Eve}, \text{David})$
- h_2 : $\text{child}(\text{David}, \text{Eve})$
- h_3 : $\text{spouse}(\text{Alice}, \text{Bob})$
- h_4 : $\text{negspouse}(\text{Alice}, \text{Bob})$
- h_5 : $\text{child}(\text{David}, \text{Carl})$
- h_6 : $\text{negchild}(\text{David}, \text{Carl})$
- h_7 : $\text{spouse}(\text{Bob}, \text{Eve})$
- h_8 : $\text{negspouse}(\text{Bob}, \text{Eve})$

Hypothesis h_1 is obtained by sampling from F (line 5), and thus it is a valid hypothesis. Then, the hypothesis h_2 is generated by altering h_1 with the function *Alter* (line 19-22). In this example, since *child* is not symmetric, h_2 is produced using a switch of the subject and the object of h_1 to generate a false hypothesis (line 6).

Hypothesis h_3 is the outcome of rule r being triggered by facts f_3 and f_4 (line 7). In a similar fashion to h_2 , we produce h_4 (line 8).

Hypothesis h_5 is sampled from the universe of all unsatisfied positive facts having a different predicate than that of the rule body (line 9), which makes it an invalid hypothesis, as it is not found in the O . Hypothesis h_6 is the negation of h_5 , and, following CWA, it is a valid hypothesis (line 10).

Finally, hypothesis h_7 is sampled from the universe of unsatisfied rule-head atoms (line 11), and it is negated to produce hypothesis h_8 .

Overall, we obtain eight different examples represented in symbolic knowledge, where each example contains the set of generated facts F , the rule r , and a single hypothesis h_i . The following is one example in symbolic knowledge:

Example #1 (Symbolic):

- **Rule r** = $\text{child}(A, C) \wedge \text{parent}(C, B) \rightarrow \text{spouse}(A, B)$
- **Facts F** :
 - f_1 : $\text{negparent}(\text{Eve}, \text{Carl})$
 - f_2 : $\text{child}(\text{Eve}, \text{David})$
 - f_3 : $\text{parent}(\text{Carl}, \text{Bob})$
 - f_4 : $\text{child}(\text{Alice}, \text{Carl})$
- **Hypothesis h_3** : $\text{spouse}(\text{Alice}, \text{Bob})$

We then convert each example to synthetic English using a set of pre-defined templates for the facts and for the rules. Here is the above Example #1, but now rewritten in synthetic English:

⁵We show an example of a soft rule in Section G below.